

PROBABILITÉS

Hasard

Correction Exercises

Exercise 1 :

```
from random import randint
for x in range(100):
    print(randint(1,6))
```

Exercise 2 :

```
from random import randint
print(randint(1,6)+randint(1,6)+randint(1,6))
```

Exercise 3 :

```
from random import randint
for x in range(50):
    print(randint(1,6),randint(1,6),randint(1,6))
```

Exercise 4 :

```
from random import randint
for x in range(200):
    print(randint(1,8),randint(1,8))
```

Exercise 5 :

```
from random import choice
liste = [1,1,2,2,2,3,3,3,4,4,4,5,5,5,6,6,6,6,6,6]
for x in range(100):
    print(choice(liste))
```

Exercise 6 :

```
from random import random
nombre = random()
if nombre < 0.1252:
    print(1)
elif nombre < 0.2561:
    print(2)
elif nombre < 0.4718:
    print(3)
elif nombre < 0.6705:
    print(4)
elif nombre < 0.7828:
    print(5)
else:
    print(6)
```

Exercice 7 :

```
from random import randint
liste_eleves=[]
groupe=[]
indice=[]

# saisie du nombre d'élèves
nbr_eleves=int(input("Combien y a t il d'élèves dans le groupe?"))

# test de la condition de parité du nombre d élèves
pair_impair=nbr_eleves%2 # calcule le reste de la division euclidienne

# Rempli le tableau avec le nom des élèves
for i in range(nbr_eleves): # pour le nombre d élèves
    nom_eleves=input("nom de l'élève") #saisir le nom des élèves
    liste_eleves.append(nom_eleves) # on affecte les noms dans la liste
    indice.append(i) # on affecte le nombre d'indice

# Rajoute un champs en cas de nombre d'élèves impair
if pair_impair==1: # si le reste =1 alors valeur impair
    indice.append(nbr_eleves) # on ajoute un indice
    nbr_eleves=nbr_eleves+1 # +1 pour nombre d'élèves pair
    liste_eleves.append('seul') # on rajoute le champ seul

print(liste_eleves)

# Tirage au sort des groupes
nbr_groupe=nbr_eleves//2 # Calcule du nombre de groupes
print('Tirage des groupes')

for i in range(nbr_groupe):

    nb=randint(0,len(indice)-1)
    efface=indice[nb]
    indice.remove(efface)
    eleve1=liste_eleves[efface]

    nb=randint(0,len(indice)-1)
    efface=indice[nb]
    indice.remove(efface)
    eleve2=liste_eleves[efface]

    print('le groupe',i+1,' sera:',eleve1,'et',eleve2)
```

Exercice 8 :

1.
 - a. Le couple (6 ; 1) permet au joueur de gagner.
 - b. La probabilité d'apparition du couple (6 ; 8) est égale à $\frac{1}{6} \times \frac{1}{10} = \frac{1}{60}$.
2. La probabilité que le joueur gagne à ce jeu est égale à $\frac{1}{6} \times \frac{1}{10} + \frac{5}{6} \times \frac{1}{20} = \frac{1}{60} + \frac{5}{120} = \frac{7}{120}$.
- 3.

```
from random import*
def de_urne():
    d=randint(1,6)
    if d==6:
        u=randint(1,10)
    else:
        u=randint(1,20)
    if u==1:
        return "Le joueur a gagné"
    else:
        return "Le joueur a perdu"
```

Exercice 9 :

```
from random import randint
def affichage():
    resultat=[]
    return [randint(1,9), randint(0,9), randint(0,9)]

def gain(partie):
    if partie[0]==partie[1] and partie[0]==partie[2]:
        return 50
    elif partie[1]==partie[0]+1 and partie[2]==partie[1]+1:
        return 20
    else:
        return 0

def gain_moyen(n):
    total=0
    for i in range(n):
        total+=gain(affichage())
    return total/n
```

Exercice 10 :

1. Cette probabilité est égale au quotient de l'aire du quart de disque par l'aire du carré, c'est-à-

dire : $p = \frac{\frac{\pi \times 1^2}{4}}{1^2} = \frac{\pi}{4}$.

2.

- a. Les coordonnées du point $M(x; y)$ doivent vérifier les inégalités suivantes :

$$0 \leq x \leq 1 \quad \text{et} \quad 0 \leq y \leq 1.$$

- b. Les coordonnées du point $M(x; y)$ doivent vérifier l'inégalité suivante :

$$x^2 + y^2 < 1$$

- c. La variable C correspond au nombre de points total présents à l'intérieur du quart de disque A .

- d. En sortie, on peut s'attendre à une valeur environ égale à $\frac{\pi}{4} \approx 0,785$.

- e. Lorsque la valeur de N augmente, on peut s'attendre à ce que le résultat renvoyé soit encore plus précis.

f.

```

from random import*
def monte_carlo():
    n=1000
    c=0
    for i in range(n):
        x=random()
        y=random()
        if x**2+y**2 < 1:
            c+=1
    return c/n

```

g.

```

from random import*
def monte_carlo():
    n=1000
    c=0
    for i in range(n):
        x=random()
        y=random()
        if x**2+y**2 < 1:
            c+=1
    return 4*c/n

```